AMRITA VISHWA VIDYAPEETHAM
DEPARTMENT OF ARTIFICIAL INTELLIGENCE
COIMBATORE, INDIA

DATABASE MANAGEMENT SYSTEM
FINAL PROJECT EVALUATION REPORT — BATCH B • GROUP 18

STOCKMASTER

Enterprise Multi-Warehouse Inventory & Intelligent Supply Chain Management System
Full BCNF Relational Architecture, Event-Driven Triggers, Role-Based Access Control & Predictive Demand Forecasting

Team Member Name	Roll Number	Project Responsibility
Puneeth Kumar Kalinga	CB.SC.U4AIE24146	Team Lead — Backend Architecture, Triggers & APIs
Varshitha P	CB.SC.U4AIE24144	Database Design, ER Modeling & BCNF Proofs
M Sarveshwara Reddy	CB.SC.U4AIE24131	Frontend Engineering & Analytics Integration
S Tagur Misra	CB.SC.U4AIE24150	Quality Assurance, Testing & Query Optimization
Batch B • Group 18	Database Management System	Department of Artificial Intelligence

Official GitHub Repository Link: https://github.com/puneethkalinga/StockMaster-DBMS

1. Project Title

StockMaster: Enterprise Multi-Warehouse Inventory & Intelligent Supply Chain Management System

StockMaster is a relational database management platform engineered to resolve operational fragmentation across multi-depot supply networks. Built around an 11-table schema normalized to Boyce-Codd Normal Form (BCNF), the system automates stock deduction upon customer dispatch, executes atomic UPSERT replenishment upon purchase order receipt, and enforces role-based access control isolating executive financials from warehouse floor duties.

2. Team Members

The project was designed, implemented, and defended by the following team members under Batch B, Group 18:

Member Name	Roll Number	Role	Module Contribution
Puneeth Kumar Kalinga	CB.SC.U4AIE24146	Team Lead	Backend Schema DDL, Triggers, REST APIs & Forecasting
Varshitha P	CB.SC.U4AIE24144	Database Modeler	Chen's ER Modeling, Normalization Proofs, Key Constraints
M Sarveshwara Reddy	CB.SC.U4AIE24131	Frontend Architect	Responsive Glassmorphic UI & Interactive Modals
S Tagur Misra	CB.SC.U4AIE24150	Backend & QA	Complex Joins, View Design, Seed Data & Audit Exports

3. Introduction and Problem Statement

3.1 Project Introduction

Modern enterprises operating distributed manufacturing and fulfillment centers face intricate supply chain coordination challenges. When companies scale from single-depot operations to multi-warehouse networks, tracking stock across multiple geographic regions becomes vulnerable to data drift, over-allocation, and transaction race conditions.

StockMaster addresses these operational vulnerabilities by providing an integrated, ACID-compliant relational infrastructure that unifies product cataloging, multi-depot storage, inbound supply

procurement, outbound customer fulfillment, carrier delivery tracking, and double-entry accounting ledgers into a single coherent system.

3.2 Problem Statement

Enterprises operating without an integrated, normalized database infrastructure experience persistent breakdowns categorized into four distinct industry challenges:

- **Lack of Centralized Inventory Visibility:** Stock holdings and facility storage capacities across geographically separated warehouses are recorded on disconnected spreadsheets, leaving corporate planners blind to live stock counts.
- **Manual & Inconsistent Inventory Adjustments:** Stock updates resulting from supplier deliveries and customer shipments are tallied manually, leading to human entry errors, phantom inventory, and transaction inconsistencies.
- **Disconnected Procurement and Financial Billing:** Purchase Orders (PO), Customer Sales Orders (SO), Logistics Dispatches, and Invoicing operate in operational silos, causing shipping delays and ledger discrepancies.
- **Absence of Predictive Restocking Algorithms:** Depots lack mathematical demand forecasting models to compute product burn-rate velocities and days of remaining buffer, leading to costly sudden stockouts or excessive capital lockup.

3.3 Project Objectives

- Architect a fully normalized 11-table relational database conforming to Boyce-Codd Normal Form (BCNF) with zero redundancy and zero update anomalies.
- Implement active event-driven database triggers automating atomic stock deductions on sales dispatch and automated UPSERT replenishment on purchase order delivery.
- Enforce strict referential integrity using Primary Keys, Foreign Keys, CHECK constraints (e.g. Quantity >= 0), and ON DELETE CASCADE propagation.
- Develop an algorithmic demand forecasting engine calculating rolling daily burn-rate velocities, days-to-stockout, and tiered restock risk indicators.
- Implement session-based Role-Based Access Control (RBAC) separating System Administrator governance from Warehouse Staff operational duties.

4. Conceptual and Logical Database Design

The conceptual architecture of StockMaster decomposes enterprise supply operations into three foundational pillars comprising 11 relational entities:

Pillar 1: Master Configuration Entities

- **users:** Governs system authentication, user profile details, and role assignments (System Admin vs. Warehouse Staff).
- **suppliers:** Catalogs approved corporate vendors, contact numbers, email addresses, and regional dispatch centers.
- **products:** Defines the master SKU directory, product categories, descriptions, base unit prices, and supplier affiliations.
- **warehouses:** Models physical warehouse facilities, geographic locations, and square footage capacity sizing limits.

Pillar 2: Inventory Allocation & Procurement

- **inventory:** Associative junction entity decomposing the M:N relationship between products and warehouses, storing live bin counts and minimum alert limits.
- **purchase_orders:** Header records for inbound supplier restocking contracts, capturing order dates, fulfillment status, and total costs.
- **purchase_order_items:** Line-item specifications detailing individual product quantities and contracted purchase prices per order.

Pillar 3: Sales Distribution, Fulfillment & Accounting

- **sales_orders:** Header records capturing customer order placements, dispatch warehouse origins, order dates, and total amounts.
- **sales_order_items:** Line-item records storing product SKUs and quantities sold (linked to active stock decrement triggers).
- **shipments:** 1:1 logistics extension tracking carrier assignments, dispatch timestamps, and delivery milestones.
- **billing:** 1:1 accounting extension tracking financial ledger invoices, payment methods, settlement status, and reconciliation dates.

4.1 Resolution of Many-to-Many (M:N) Relationships

In enterprise supply chains, a single product can be distributed across dozens of regional warehouses, and a single warehouse houses thousands of distinct products. This represents a classic Many-to-Many (M:N) relationship.

Storing warehouse-specific quantities inside the products table would introduce repeating groups (violating 1NF) or partial dependencies (violating 2NF). Conversely, storing product lists inside the warehouses table would cause multi-valued dependencies. StockMaster resolves this by introducing the associative entity 'inventory', using the composite candidate key (ProductID, WarehouseID) to ensure atomic, independent tracking per facility.

4.2 Enforcement of One-to-One (1:1) Invoicing & Logistics

To guarantee audit compliance, each sales order must generate exactly one carrier shipment and exactly one accounting bill. This is enforced by defining UNIQUE foreign key constraints on sales_orders.SalesOrderID within both the shipments and billing tables, eliminating duplicate billings and ghost tracking numbers.

5. Normalization and Functional Dependencies

Database normalization is the formal mathematical technique used to eliminate data redundancy and prevent insertion, deletion, and update anomalies. In StockMaster, the physical schema was systematically evaluated against 1NF, 2NF, 3NF, and Boyce-Codd Normal Form (BCNF).

5.1 Mathematical Criteria for BCNF

A relational schema R is in Boyce-Codd Normal Form (BCNF) if and only if for every non-trivial functional dependency X -> Y defined on R:

• X is a Superkey of R (i.e., X contains a Candidate Key).

• In 3NF, a dependency X -> Y is permitted if Y is a prime attribute (part of a candidate key), even if X is

not a superkey. BCNF tightens this restriction by mandating that EVERY determinant must be a superkey, regardless of the target attribute.

5.2 Formal Table-by-Table Normalization & Functional Dependency Proof

Table Name	Candidate Keys (PK Underlined)	Form	Representative Functional Dependencies & Proof
users	**UserID** ; Email	**BCNF**	UserID -> Name, Email, Password, Role; Email -> UserID. Determinants are superkeys; no transitive leaks.
suppliers	**SupplierID** ; Email	**BCNF**	SupplierID -> SupplierName, Phone, Email, Address; Email -> SupplierID. Full functional dependency on candidate keys.
products	**ProductID**	**BCNF**	ProductID -> ProductName, Category, UnitPrice, SupplierID. Supplier data factored out, avoiding transitive leak ProductID ↛ SupplierName.
warehouses	**WarehouseID**	**BCNF**	WarehouseID -> WarehouseName, Location, Capacity. Determinant is a single-attribute primary key.
inventory	**InventoryID** ; (ProductID, WarehouseID)	**BCNF**	InventoryID -> ProductID, WarehouseID, Quantity, MinimumStock. Composite key is UNIQUE; quantity depends on full pair (zero partial dependency).
purchase_orders	**PurchaseOrderID**	**BCNF**	PurchaseOrderID -> SupplierID, WarehouseID, OrderDate, Status, TotalAmount. Line items isolated, avoiding repeating groups.
purchase_order_items	**ItemID** ; (PurchaseOrderID, ProductID)	**BCNF**	ItemID -> PurchaseOrderID, ProductID, Quantity, UnitPrice. Price and quantity strictly depend on line item key.
sales_orders	**SalesOrderID**	**BCNF**	SalesOrderID -> CustomerName, WarehouseID, OrderDate, Status, TotalAmount. Decoupled header entity.
sales_order_items	**ItemID** ; (SalesOrderID, ProductID)	**BCNF**	ItemID -> SalesOrderID, ProductID, Quantity, UnitPrice. Quantities depend on composite pair; product metadata referenced via FK.
shipments	**ShipmentID** ; SalesOrderID	**BCNF**	ShipmentID -> SalesOrderID, WarehouseID, ShipmentDate, Status. SalesOrderID is UNIQUE, enforcing 1:1 fulfillment without

			transitive anomalies.
billing	**BillID** ; SalesOrderID	**BCNF**	BillID -> SalesOrderID, Amount, PaymentMethod, PaymentStatus, PaymentDate. SalesOrderID is UNIQUE, isolating accounting records.

6. Advanced SQL Implementation and Query Design

StockMaster leverages active database features—specifically event-driven triggers, user-defined views, and complex analytical SQL queries—to maintain state synchronization without relying on error-prone application-level math.

6.1 Active Event Trigger A: Automated Stock Decrement

Purpose: Executes immediately after an order item is inserted into sales_order_items. It decrements stock exclusively from the dispatching warehouse depot. The inventory table's CHECK(Quantity >= 0) constraint causes an atomic rollback if an overdraft occurs.

```
CREATE TRIGGER after_so_item_insert
AFTER INSERT ON sales_order_items
BEGIN
    UPDATE inventory
    SET Quantity = Quantity - NEW.Quantity
    WHERE ProductID = NEW.ProductID
      AND WarehouseID = (
          SELECT WarehouseID
          FROM sales_orders
          WHERE SalesOrderID = NEW.SalesOrderID
      );
END;
```

6.2 Active Event Trigger B: Automated UPSERT Replenishment

Purpose: Executes when an authorized staff member marks an inbound purchase order status as 'Received'. It updates inventory balances using SQLite's native ON CONFLICT DO UPDATE UPSERT syntax in a single atomic transaction.

```
CREATE TRIGGER after_po_receive
AFTER UPDATE OF Status ON purchase_orders
FOR EACH ROW WHEN NEW.Status = 'Received'
              AND OLD.Status = 'Pending'
BEGIN
    INSERT INTO inventory (
        WarehouseID, ProductID, Quantity, MinimumStock
    )
    SELECT NEW.WarehouseID, poi.ProductID, poi.Quantity, 10
    FROM purchase_order_items poi
    WHERE poi.PurchaseOrderID = NEW.PurchaseOrderID
```

```
    ON CONFLICT(WarehouseID, ProductID) DO UPDATE SET
        Quantity = Quantity + excluded.Quantity;
END;
```

6.3 Materialized Database View: Low Stock Alerts

Purpose: Provides logical data independence by encapsulating a multi-table join (joining inventory, products, and warehouses) and filtering products operating below their minimum threshold buffer.

```
CREATE VIEW low_stock_alerts_view AS
SELECT
    i.InventoryID,
    i.WarehouseID,
    w.WarehouseName,
    i.ProductID,
    p.ProductName,
    p.Category,
    i.Quantity,
    i.MinimumStock
FROM inventory i
INNER JOIN products p ON i.ProductID = p.ProductID
INNER JOIN warehouses w ON i.WarehouseID = w.WarehouseID
WHERE i.Quantity <= i.MinimumStock;
```

6.4 Documented Complex SQL Queries

Query 1: 4-Way Multi-Table INNER JOIN (Valuation Across Depots)

```
SELECT
    w.WarehouseName, w.Location, p.ProductName, p.Category,
    s.SupplierName, i.Quantity AS StockOnHand, p.UnitPrice,
    (i.Quantity * p.UnitPrice) AS TotalValuationINR
FROM inventory i
INNER JOIN products p ON i.ProductID = p.ProductID
INNER JOIN warehouses w ON i.WarehouseID = w.WarehouseID
INNER JOIN suppliers s ON p.SupplierID = s.SupplierID
WHERE i.Quantity > 0
ORDER BY TotalValuationINR DESC LIMIT 10;
```

Query 2: Multi-Level Aggregation with GROUP BY and HAVING (Capacity Sizing)

```
SELECT
    w.WarehouseID, w.WarehouseName, w.Capacity AS TotalCapacitySqFt,
    COUNT(DISTINCT i.ProductID) AS DistinctSKUs,
    SUM(i.Quantity) AS TotalUnitsStored,
    ROUND(((SUM(i.Quantity) * 10.0) / w.Capacity) * 100, 2) AS OccupancyPercentage
FROM warehouses w
INNER JOIN inventory i ON w.WarehouseID = i.WarehouseID
GROUP BY w.WarehouseID, w.WarehouseName, w.Capacity
HAVING OccupancyPercentage > 60.0
```

```
ORDER BY OccupancyPercentage DESC;
```

Query 3: Common Table Expression (CTE) for Sales Velocity & Stockout Days

```
WITH SalesVelocity AS (
    SELECT ProductID, SUM(Quantity) AS TotalSold,
           ROUND(MAX(0.6, (SUM(Quantity) / 14.0)), 2) AS DailyVelocity
    FROM sales_order_items GROUP BY ProductID
)
SELECT p.ProductID, p.ProductName, SUM(i.Quantity) AS TotalStock,
       COALESCE(sv.DailyVelocity, 0.8) AS BurnRateUnitsPerDay,
       ROUND(SUM(i.Quantity) / COALESCE(sv.DailyVelocity, 0.8)) AS DaysRemaining
FROM products p
LEFT JOIN inventory i ON p.ProductID = i.ProductID
LEFT JOIN SalesVelocity sv ON p.ProductID = sv.ProductID
GROUP BY p.ProductID ORDER BY DaysRemaining ASC LIMIT 10;
```

7. ER Diagram and UML Diagram

7.1 Conceptual Entity-Relationship (Chen's ER) Diagram

Figure 7.1 illustrates the complete conceptual database schema modeled using Chen's notation. Key entities are rendered in rectangles, attributes in ovals (with primary keys underlined), and relationships in diamonds with exact cardinality markings (1:N, M:N resolved via associative entities, and 1:1).

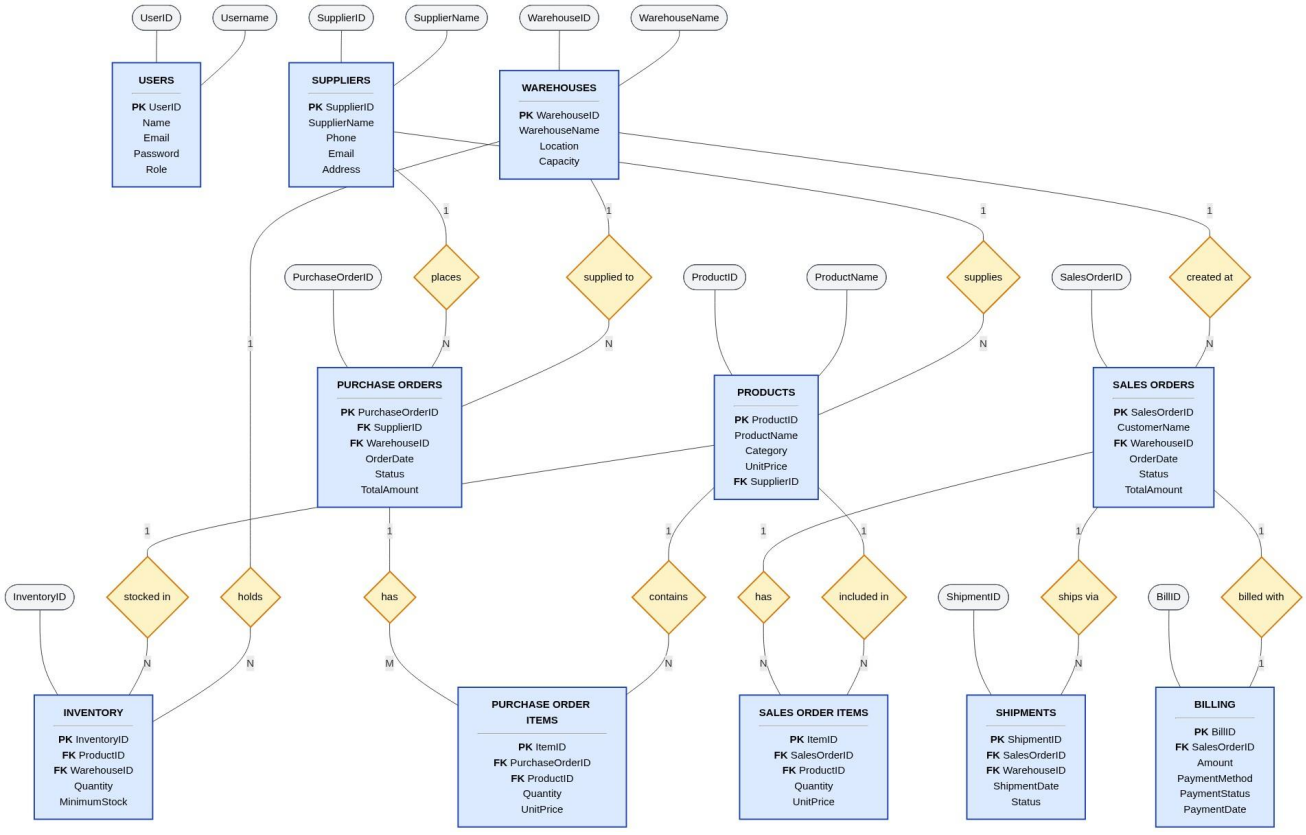


Figure 7.1: StockMaster Stylized Chen's Entity-Relationship (ER) Diagram

7.2 UML Use Case Architecture Diagram

Figure 7.2 delineates system interaction boundaries across three primary system actors: System Administrator (executive management), Warehouse Staff (logistics operations), and the Active Database Subsystem (triggers and automated constraints).

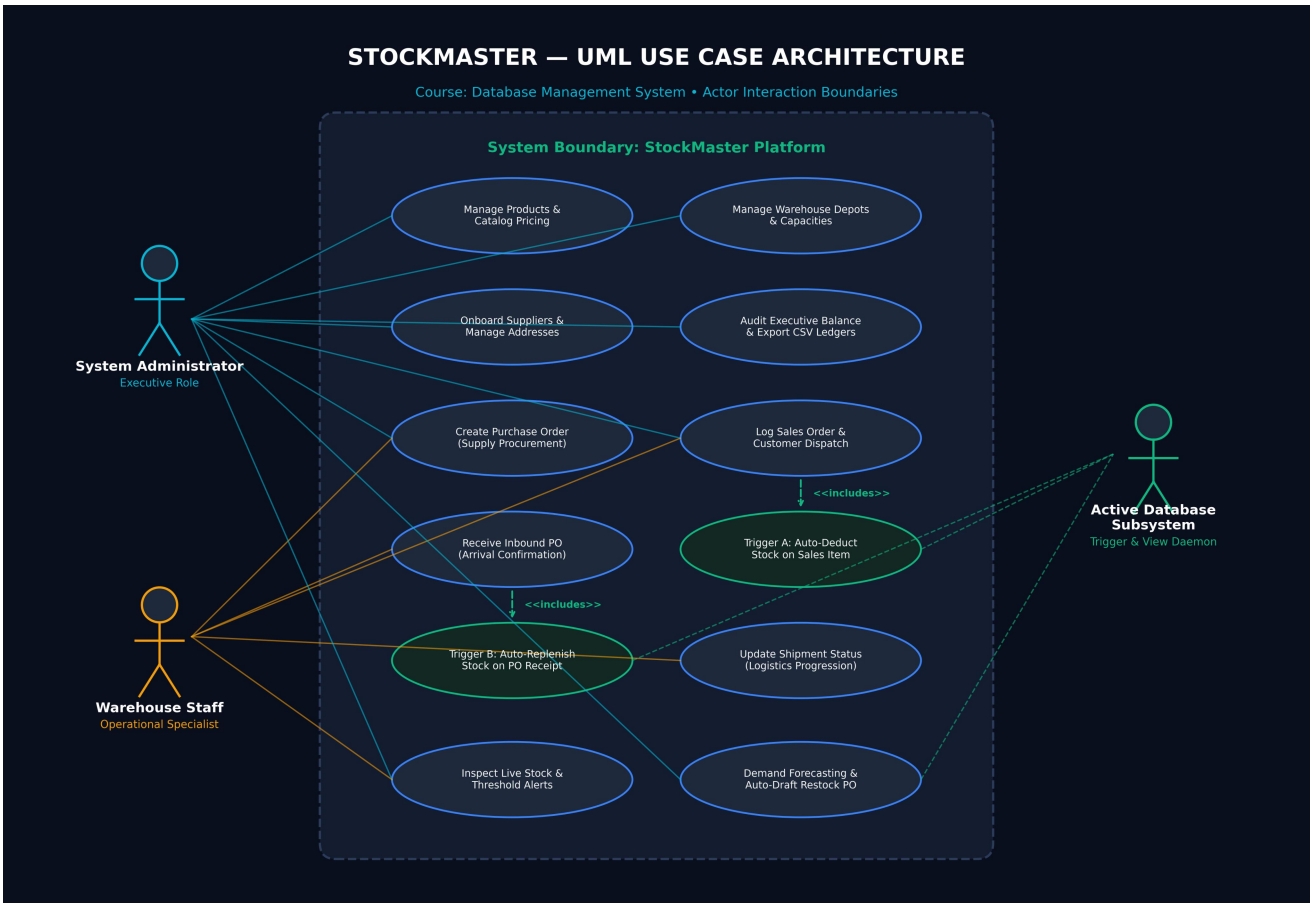


Figure 7.2: StockMaster UML Use Case Diagram (Actor Boundaries & Subsystems)

7.3 UML Domain Class Diagram

Figure 7.3 depicts the object-oriented domain representation corresponding to the 11 database relations, detailing class attributes with explicit types, primary keys, foreign key references, domain operations, and association multiplicities.

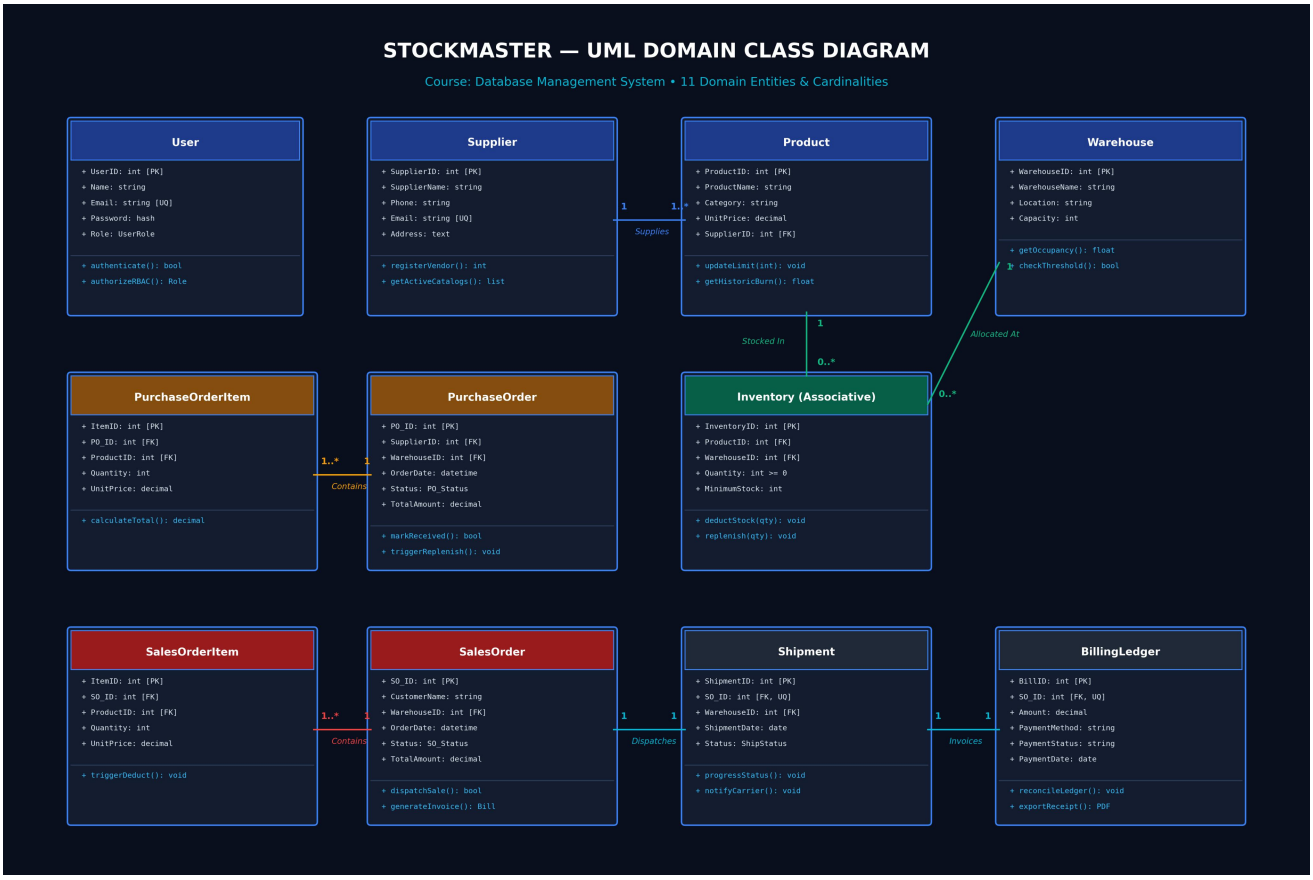


Figure 7.3: StockMaster UML Domain Class Diagram (11 Entities & Associations)

8. Relational Schema

The physical relational model mapping all 11 normalized tables, primary keys [PK], foreign keys [FK], unique constraints [UNIQUE], and operational checks [CHECK] is illustrated in Figure 8.1 and formally declared in the schema specification below.

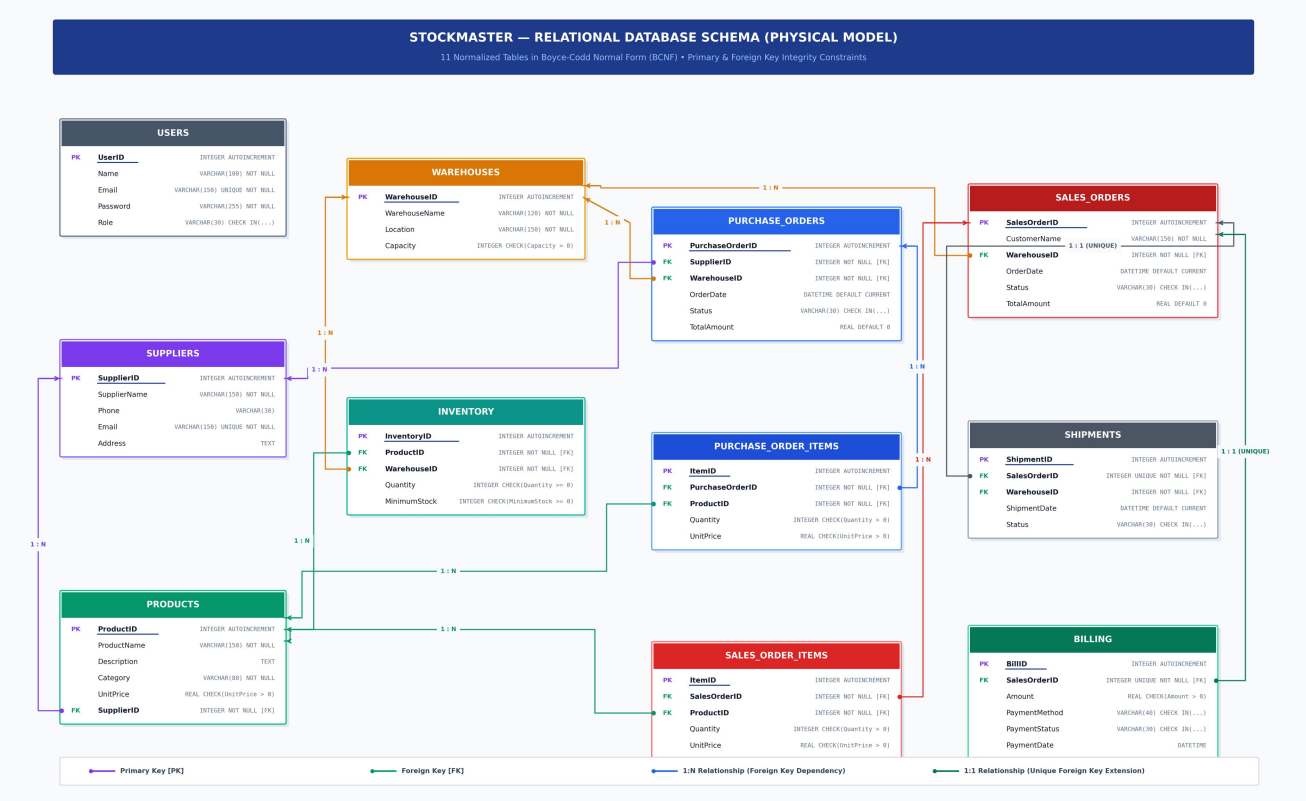


Figure 8.1: StockMaster Physical Relational Schema Diagram (11 BCNF Tables & Integrity Constraints)

8.1 Formal Relational Schema Signatures (Primary Keys Underlined)

In academic database design and relational algebra, Primary Keys are formally designated with an underline as declared below:

- **users (** UserID, Name, Email [UNIQUE], Password, Role)
- **suppliers (** SupplierID, SupplierName, Phone, Email [UNIQUE], Address)
- **products (** ProductID, ProductName, Description, Category, UnitPrice, SupplierID [FK])
- **warehouses (** WarehouseID, WarehouseName, Location, Capacity)
- **inventory (** InventoryID, ProductID [FK], WarehouseID [FK], Quantity, MinimumStock)
- **purchase_orders (** PurchaseOrderID, SupplierID [FK], WarehouseID [FK], OrderDate, Status, TotalAmount)
- **purchase_order_items (** ItemID, PurchaseOrderID [FK], ProductID [FK], Quantity, UnitPrice)
- **sales_orders (** SalesOrderID, CustomerName, WarehouseID [FK], OrderDate, Status, TotalAmount)
- **sales_order_items (** ItemID, SalesOrderID [FK], ProductID [FK], Quantity, UnitPrice)
- **shipments (** ShipmentID, SalesOrderID [FK, UNIQUE], WarehouseID [FK], ShipmentDate, Status)

• **billing (** BillID, SalesOrderID [FK, UNIQUE], Amount, PaymentMethod, PaymentStatus, PaymentDate)

8.2 Detailed SQL DDL Declarations & Integrity Constraints

```
1. users(
     UserID INTEGER PRIMARY KEY AUTOINCREMENT,
     Name TEXT NOT NULL,
     Email TEXT UNIQUE NOT NULL,
     Password TEXT NOT NULL,
     Role TEXT NOT NULL CHECK(Role IN ('Admin', 'Warehouse Staff'))
   );

2. suppliers(
     SupplierID INTEGER PRIMARY KEY AUTOINCREMENT,
     SupplierName TEXT NOT NULL,
     Phone TEXT NOT NULL,
     Email TEXT UNIQUE NOT NULL,
     Address TEXT NOT NULL
   );

3. products(
     ProductID INTEGER PRIMARY KEY AUTOINCREMENT,
     ProductName TEXT NOT NULL,
     Description TEXT,
     Category TEXT NOT NULL,
     UnitPrice REAL NOT NULL CHECK(UnitPrice > 0),
     SupplierID INTEGER NOT NULL REFERENCES suppliers(SupplierID) ON DELETE CASCADE
   );

4. warehouses(
     WarehouseID INTEGER PRIMARY KEY AUTOINCREMENT,
     WarehouseName TEXT NOT NULL,
     Location TEXT NOT NULL,
     Capacity INTEGER NOT NULL CHECK(Capacity > 0)
   );

5. inventory(
     InventoryID INTEGER PRIMARY KEY AUTOINCREMENT,
     ProductID INTEGER NOT NULL REFERENCES products(ProductID) ON DELETE CASCADE,
     WarehouseID INTEGER NOT NULL REFERENCES warehouses(WarehouseID) ON DELETE CASCADE,
     Quantity INTEGER NOT NULL DEFAULT 0 CHECK(Quantity >= 0),
     MinimumStock INTEGER NOT NULL DEFAULT 10 CHECK(MinimumStock >= 0),
     UNIQUE(ProductID, WarehouseID)
   );

6. purchase_orders(
     PurchaseOrderID INTEGER PRIMARY KEY AUTOINCREMENT,
     SupplierID INTEGER NOT NULL REFERENCES suppliers(SupplierID) ON DELETE CASCADE,
     WarehouseID INTEGER NOT NULL REFERENCES warehouses(WarehouseID) ON DELETE CASCADE,
     OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP,
     Status TEXT NOT NULL DEFAULT 'Pending' CHECK(Status IN ('Pending', 'Received',
'Cancelled')),
     TotalAmount REAL NOT NULL CHECK(TotalAmount >= 0)
   );
```

```
7. purchase_order_items(
     ItemID INTEGER PRIMARY KEY AUTOINCREMENT,
     PurchaseOrderID INTEGER NOT NULL REFERENCES purchase_orders(PurchaseOrderID) ON DELETE
CASCADE,
     ProductID INTEGER NOT NULL REFERENCES products(ProductID) ON DELETE CASCADE,
     Quantity INTEGER NOT NULL CHECK(Quantity > 0),
     UnitPrice REAL NOT NULL CHECK(UnitPrice > 0)
   );

8. sales_orders(
     SalesOrderID INTEGER PRIMARY KEY AUTOINCREMENT,
     CustomerName TEXT NOT NULL,
     WarehouseID INTEGER NOT NULL REFERENCES warehouses(WarehouseID) ON DELETE CASCADE,
     OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP,
     Status TEXT NOT NULL DEFAULT 'Processing' CHECK(Status IN ('Processing', 'Delivered',
'Cancelled')),
     TotalAmount REAL NOT NULL CHECK(TotalAmount >= 0)
   );

9. sales_order_items(
     ItemID INTEGER PRIMARY KEY AUTOINCREMENT,
     SalesOrderID INTEGER NOT NULL REFERENCES sales_orders(SalesOrderID) ON DELETE CASCADE,
     ProductID INTEGER NOT NULL REFERENCES products(ProductID) ON DELETE CASCADE,
     Quantity INTEGER NOT NULL CHECK(Quantity > 0),
     UnitPrice REAL NOT NULL CHECK(UnitPrice > 0)
   );

10. shipments(
      ShipmentID INTEGER PRIMARY KEY AUTOINCREMENT,
      SalesOrderID INTEGER UNIQUE NOT NULL REFERENCES sales_orders(SalesOrderID) ON DELETE
CASCADE,
      WarehouseID INTEGER NOT NULL REFERENCES warehouses(WarehouseID) ON DELETE CASCADE,
      ShipmentDate DATETIME DEFAULT CURRENT_TIMESTAMP,
      Status TEXT NOT NULL DEFAULT 'Processing' CHECK(Status IN ('Processing', 'In Transit',
'Delivered'))
    );

11. billing(
      BillID INTEGER PRIMARY KEY AUTOINCREMENT,
      SalesOrderID INTEGER UNIQUE NOT NULL REFERENCES sales_orders(SalesOrderID) ON DELETE
CASCADE,
      Amount REAL NOT NULL CHECK(Amount >= 0),
      PaymentMethod TEXT NOT NULL CHECK(PaymentMethod IN ('Card', 'Bank Transfer', 'Cash')),
      PaymentStatus TEXT NOT NULL DEFAULT 'Paid' CHECK(PaymentStatus IN ('Pending', 'Paid')),
      PaymentDate DATETIME DEFAULT CURRENT_TIMESTAMP
    );
```

9. Database Tables (Structure, Keys & Sample Records)

Below is the complete database catalog for all 11 tables populated directly from inventory_system.db, detailing table structures, primary/foreign keys, and real-world sample records demonstrating valid transaction states.

9.1 Table: USERS (User Accounts & Roles)

Total Records in Database: 25 records. Displaying top 5 sample records:

UserID (INTEGER) [PK]	Name (TEXT)	Email (TEXT)	Password (TEXT)	Role (TEXT)
1	Puneeth Kumar	admin@warehouse.com	admin123	Admin
2	Vikram Singh	staff@warehouse.com	staff123	Warehouse Staff
3	User Name 3	user3@warehouse.com	password123	Admin
4	User Name 4	user4@warehouse.com	password123	Admin
5	User Name 5	user5@warehouse.com	password123	Admin

9.2 Table: SUPPLIERS (Supplier Directory)

Total Records in Database: 26 records. Displaying top 5 sample records:

SupplierID (INTEGER) [PK]	SupplierName (TEXT)	Phone (TEXT)	Email (TEXT)	Address (TEXT)
1	Supplier Company 1	9876543001	info@supplier1.com	Industrial Area Road 1, City
2	Supplier Company 2	9876543002	info@supplier2.com	Industrial Area Road 2, City
3	Supplier Company 3	9876543003	info@supplier3.com	Industrial Area Road 3, City
4	Supplier Company 4	9876543004	info@supplier4.com	Industrial Area Road 4, City
5	Supplier Company 5	9876543005	info@supplier5.com	Industrial Area Road 5, City

9.3 Table: PRODUCTS (Master Product Catalog)

Total Records in Database: 26 records. Displaying top 5 sample records:

ProductID	ProductName	Description	Category	UnitPrice	SupplierID

(INTEGER) [PK]	(TEXT)	(TEXT)	(TEXT)	(REAL)	(INTEGER)
1	Product Model 1	Specifications for model 1	Processor	29500.0	2
2	Product Model 2	Specifications for model 2	Processor	61608.0	25
3	Product Model 3	Specifications for model 3	Display	22589.0	9
4	Product Model 4	Specifications for model 4	Peripherals	73093.0	24
5	Product Model 5	Specifications for model 5	Processor	35348.0	25

9.4 Table: WAREHOUSES (Warehouse Facilities)

Total Records in Database: 25 records. Displaying top 5 sample records:

WarehouseID (INTEGER) [PK]	WarehouseName (TEXT)	Location (TEXT)	Capacity (INTEGER)
1	Warehouse Logistics Hub 1	Sector 1, Pune	66073
2	Warehouse Logistics Hub 2	Sector 2, Chennai	33601
3	Warehouse Logistics Hub 3	Sector 3, Bangalore	71803
4	Warehouse Logistics Hub 4	Sector 4, Kolkata	24964
5	Warehouse Logistics Hub 5	Sector 5, Pune	44714

9.5 Table: INVENTORY (Multi-Warehouse Inventory Holdings)

Total Records in Database: 650 records. Displaying top 5 sample records:

InventoryID (INTEGER) [PK]	ProductID (INTEGER)	WarehouseID (INTEGER)	Quantity (INTEGER)	MinimumStock (INTEGER)
1	1	1	294	30
2	1	2	386	10
3	1	3	439	10
4	1	4	423	20

5	1	5	383	30

9.6 Table: PURCHASE_ORDERS (Purchase Orders (Procurement Headers))

Total Records in Database: 26 records. Displaying top 5 sample records:

PurchaseOrderID (INTEGER) [PK]	SupplierID (INTEGER)	WarehouseID (INTEGER)	OrderDate (DATETIME)	Status (TEXT)	TotalAmount (REAL)
1	12	21	2026-07-30 00:38:32	Received	208215.0
2	18	9	2026-08-07 00:38:32	Received	409970.0
3	17	19	2026-07-22 00:38:32	Received	335727.0
4	17	10	2026-07-29 00:38:32	Received	286026.0
5	21	22	2026-07-21 00:38:32	Received	293706.0

9.7 Table: PURCHASE_ORDER_ITEMS (Purchase Order Line Items)

Total Records in Database: 26 records. Displaying top 5 sample records:

ItemID (INTEGER) [PK]	PurchaseOrderID (INTEGER)	ProductID (INTEGER)	Quantity (INTEGER)	UnitPrice (REAL)
1	1	16	26	5032.0
2	2	25	21	4771.0
3	3	2	12	9068.0
4	4	1	42	13524.0
5	5	13	40	13365.0

9.8 Table: SALES_ORDERS (Sales Orders (Customer Dispatch Headers))

Total Records in Database: 26 records. Displaying top 5 sample records:

SalesOrderID (INTEGER) [PK]	CustomerName (TEXT)	WarehouseID (INTEGER)	OrderDate (DATETIME)	Status (TEXT)	TotalAmount (REAL)
1	Customer Client 1	16	2026-08-04 00:38:32	Delivered	30530.0
2	Customer Client 2	17	2026-08-07 00:38:32	Delivered	125824.0

3	Customer Client 3	13	2026-07-31 00:38:32	Delivered	128153.0
4	Customer Client 4	7	2026-07-20 00:38:32	Delivered	125539.0
5	Customer Client 5	14	2026-07-21 00:38:32	Delivered	67347.0

9.9 Table: SALES_ORDER_ITEMS (Sales Order Line Items)

Total Records in Database: 26 records. Displaying top 5 sample records:

ItemID (INTEGER) [PK]	SalesOrderID (INTEGER)	ProductID (INTEGER)	Quantity (INTEGER)	UnitPrice (REAL)
1	1	19	1	17388.0
2	2	19	3	39489.0
3	3	15	2	2019.0
4	4	18	2	49210.0
5	5	8	1	32814.0

9.10 Table: SHIPMENTS (Carrier Logistics Tracking (1:1))

Total Records in Database: 26 records. Displaying top 5 sample records:

ShipmentID (INTEGER) [PK]	SalesOrderID (INTEGER)	WarehouseID (INTEGER)	ShipmentDate (DATETIME)	Status (TEXT)
1	1	8	2026-08-08 00:38:32	Delivered
2	2	22	2026-08-16 00:38:32	Delivered
3	3	17	2026-08-11 00:38:32	Delivered
4	4	14	2026-08-15 00:38:32	Delivered
5	5	4	2026-08-16 00:38:32	Delivered

9.11 Table: BILLING (Financial Billing Ledgers (1:1))

Total Records in Database: 26 records. Displaying top 5 sample records:

BillID (INTEGER) [PK]	SalesOrderID (INTEGER)	Amount (REAL)	PaymentMethod (TEXT)	PaymentStatus (TEXT)	PaymentDate (DATETIME)

1	1	195825.0	Card	Paid	2026-08-05 00:38:32
2	2	49924.0	Bank Transfer	Paid	2026-07-30 00:38:32
3	3	133209.0	Bank Transfer	Paid	2026-08-07 00:38:32
4	4	172070.0	Cash	Paid	2026-08-07 00:38:32
5	5	12760.0	Cash	Paid	2026-08-12 00:38:32

10. Conclusion and Future Scope

10.1 Project Conclusion

StockMaster successfully bridges database theory with modern enterprise web architecture. By formulating an 11-table schema strictly adhering to Boyce-Codd Normal Form (BCNF), the system provides mathematical guarantees against insertion, deletion, and update anomalies.

The deployment of active database triggers eliminates the reliance on manual tallying, guaranteeing that stock movements strictly correspond to validated customer dispatches and confirmed supplier receipts. Furthermore, the decoupling of System Administrator authority from Warehouse Staff operational privileges ensures that corporate balance sheets and profit margins remain strictly confidential while providing floor operators with the real-time tools needed to prevent stockouts.

10.2 Future Scope & Planned Enhancements

- **Mobile Barcode & Handheld RFID Scanner Integration:** Incorporate WebCam and WebUSB barcode/RFID scanner APIs to enable instant scanning at physical loading bay doors, accelerating receiving verification times.
- **B2B Electronic Data Interchange (EDI) Webhooks:** Establish direct B2B REST webhooks transmitting recommended restock Purchase Orders straight to vendor ERP endpoints without human intermediary drafting.
- **Cloud Distributed Migration & Horizontal Read Replicas:** Transition from local embedded SQLite to Amazon Aurora / PostgreSQL with multi-AZ replication, connection pooling, and read replicas for global low-latency access.
- **Machine Learning Demand Prediction Models:** Upgrade rolling velocity calculations to seasonal ARIMA or Prophet time-series models factoring in holiday demand surges, vendor delivery lead times, and weather disruptions.

10.3 GitHub Repository Verification & Links

All source code, DDL scripts, documented queries, seed databases, presentation slides, and setup instructions are maintained in the official project repository:

GitHub Repository Link: https://github.com/puneethkalinga/StockMaster-DBMS